

Course Syllabus

725103 Data Structures and Algorithms for Business

คณะบริหารธุรกิจ | มหาวิทยาลัยเอเชียอาคเนย์ | ภาคเรียนที่ 2 ปีการศึกษา 2568

อาจารย์ผู้สอน: ปพนธีร์ แย้มโชติ | phaphontee@sau.ac.th
(โดยปกติติดต่อผ่านแชท Microsoft Teams (Phaphontee Yamchote) จะตอบเร็วสุด)

1 ภาพรวมรายวิชา (Course Overview)

รายวิชานี้มุ่งพัฒนาทักษะ การเลือกโครงสร้างข้อมูลและอัลกอริทึมให้เหมาะกับโจทย์ โดยเน้นการคิดแบบ **Visualize, Tracing, Pseudocode**:

- เห็นโจทย์ $\rightarrow$ ระบุ ADT และ operation ที่ต้องการ
- เขียน/อ่าน **pseudocode** และ **trace** ที่ละขั้น
- วิเคราะห์เวลา/พื้นที่ด้วย **Big-O** และเลือกวิธีที่เหมาะสมกับปัญหา

หัวข้อหลักตลอดภาค (พร้อมคำสำคัญภาษาอังกฤษ):

- **Algorithmic Thinking**: pseudocode, correctness (แนวคิด), best/worst case, Asymptotic Notation ($O(\cdot)$)
- **Linear Data Structures**: array, dynamic array, list, linked list, **stack**, **queue**
- **Recursion** และแนวคิด **divide-and-conquer**
- **Trees**: binary tree, traversal (BFS/DFS), BST, **trie**
- **Priority Queue & Heap**: heap operations, **Heap Sort**
- **Hash Table**: hash function, collision (เช่น chaining / open addressing)
- **Graphs**: representation, traversal, shortest path (แนวคิด), **MST** (แนวคิด)
- **Searching & Sorting**: linear/binary search, sorting ideas & complexity trade-off

1.1 โครงสร้างกิจกรรมมาตรฐานประจำสัปดาห์ (6-step + 1-optional)

1. Concept & ADT (โจทย์ต้องการ operation อะไร)
2. Visualization (วาดภาพ/แผนภาพ)
3. Pseudocode (อ่าน/เขียนขั้นตอน)
4. Trace (ไล่ค่าทีละสเต็ปจนมั่นใจ)
5. Complexity (Big-O ระดับแนวคิด)
6. Reflection (อธิบายเหตุผลการเลือกวิธี + จุดพลาดที่พบบ่อย)
7. Coding (lowest priority, self-studying: จะไม่ใช้การเขียนโค้ดเป็นการวัดผลหลัก)

1.2 แนวทางและจุดเน้นของรายวิชา

- วิชานี้ไม่เน้นการเขียนโค้ด (coding) เป็นหลัก
เน้นความเข้าใจเชิงแนวคิดในการ เลือกและอธิบายเหตุผล ของการใช้ **data structure** แต่ละแบบ เช่น ทำ **trace** จาก **pseudocode**, วาดภาพโครงสร้าง, และเปรียบเทียบประสิทธิภาพด้วย **Big-O** มากกว่าการ **implement** ให้สมบูรณ์ เหมือนรายวิชาเขียนโปรแกรม
- สอดคล้องกับบริบทของหลักสูตรคอมพิวเตอร์เพื่อธุรกิจ (ชื่อใหม่: เทคโนโลยีสารสนเทศเพื่อธุรกิจดิจิทัล)
จึงให้ความสำคัญกับ เหตุผลและการสื่อสารเชิงวิศวกรรม มากกว่า **implementation**: นักศึกษาควรตอบให้ได้ว่า “ทำไมต้องเลือกโครงสร้างนี้?” “ข้อดี/ข้อจำกัดคืออะไร?” และ “ถ้าเงื่อนไขโจทย์เปลี่ยน จะเปลี่ยนโครงสร้างหรือ **algorithm** อย่างไร?”
- อย่างไรก็ตาม รายวิชาจะเตรียมทางเลือกสำหรับผู้ที่ต้องการฝึก **implement**
ผู้สอนจะ เสนอแหล่งฝึกเพิ่มเติม (optional) เพื่อให้เมื่อนักศึกษามีความคล่องในการเขียนโปรแกรม ระหว่างหรือหลัง เรียนวิชา 725102 การเขียนโปรแกรมด้วยภาษาไพธอน นักศึกษาจะสามารถนำความรู้ในวิชานี้ไป **implement** (เช่น Python) ได้ทันที

2 ข้อมูลรายวิชา (Course Information)

2.1 หน่วยกิต/ชั่วโมงเรียนและวิชาบังคับก่อน

- หน่วยกิต: 3 (2--2--6) (บรรยาย 2 ชม./สัปดาห์, ปฏิบัติ 2 ชม./สัปดาห์, ศึกษาด้วยตนเองโดยประมาณ 6 ชม./สัปดาห์)
- วิชาบังคับก่อน (Pre-requisite): ไม่มี
- รูปแบบรายวิชา: บรรยาย + ปฏิบัติ (trace/worksheet/กิจกรรม) + แบบทดสอบย่อย

2.2 คำอธิบายรายวิชา (Course Description)

ศึกษาความรู้เกี่ยวกับโครงสร้างข้อมูลและอัลกอริทึมพื้นฐาน ทั้งเชิงเส้นและไม่เชิงเส้น เช่น array, linked list, stack, queue, tree, graph รวมถึงแนวคิดการวิเคราะห์ประสิทธิภาพของอัลกอริทึม และการประยุกต์ใช้เพื่อแก้ปัญหาอย่างมีประสิทธิภาพ

2.3 ซอฟต์แวร์/อุปกรณ์ที่ใช้ (Required Tools)

- Microsoft Teams (ประกาศรายวิชา/ส่งงาน/สื่อสาร)
- เครื่องมือช่วย visualization (เช่น Python Tutor, VisuAlgo) (ใช้เพื่ออธิบาย/ตรวจสอบความเข้าใจ)
- เครื่องเขียน/แท็บเล็ตสำหรับวาดแผนภาพและทำ trace

2.4 ชั่วโมงให้คำปรึกษา (Office Hours)

- อังคาร 13.00--16.00 น. และ เสาร์ 13.00--16.00 น. (หรือเวลาอื่นตามนัดหมายผ่าน Teams)
- อีเมล: phaphontee@sau.ac.th

3 ผลลัพธ์การเรียนรู้ (Learning Outcomes / CLOs)

เมื่อจบรายวิชา นักศึกษาควรสามารถ:

1. อธิบายแนวคิดพื้นฐานของ **data structure** และ **algorithm** พร้อมยกตัวอย่างการใช้งานได้
2. อ่าน/เขียน **pseudocode** และทำ **trace** เพื่ออธิบายการทำงานของโครงสร้างข้อมูล/อัลกอริทึมได้อย่างเป็นขั้นตอน
3. วิเคราะห์โจทย์และเลือก **ADT/โครงสร้างข้อมูล** ที่เหมาะสมกับปัญหาได้อย่างมีเหตุผล
4. ประเมินประสิทธิภาพเชิงแนวคิดด้วย **Big-O** และอธิบาย **trade-off** เวลา/หน่วยความจำได้
5. ทำงานร่วมกับผู้อื่นและสื่อสารคำอธิบายด้วยแผนภาพ/ตัวอย่าง/คำศัพท์เทคนิคอย่างถูกต้อง

4 วิธีการเรียนการสอน (Teaching & Learning)

4.1 จังหวะการเรียนรู้แบบ Visual → Trace → Reason

ในรายวิชานี้ “ทำให้เห็นภาพและ trace ได้” มาก่อน “ท่องจำ”:

- เน้นการวาดภาพ memory/โครงสร้าง
- เน้นการไล่ค่าทีละขั้นจาก pseudocode
- ปิดท้ายด้วย Big-O และการเลือกใช้ให้เหมาะสมกับสถานการณ์

4.2 โครงคาบเรียน (4 ชั่วโมง/สัปดาห์)

- ช่วงบรรยาย: แนวคิด + ภาพรวม + ตัวอย่างเล็ก
- ช่วงปฏิบัติ: worksheet/กิจกรรม + trace + quiz สั้น (เช็คความเข้าใจ)

4.3 รูปแบบงานในรายวิชา (Work Pattern)

- งานเดี่ยว: ใบงาน/การบ้าน (เน้น trace + อธิบายเหตุผล + Big-O ระดับแนวคิด)
- สอบย่อย (Quiz): 2 ครั้ง สอบ takehome 1 สัปดาห์
- งานกลุ่ม: อธิบายการเลือก DS/algorithm จากโจทย์ + นำเสนอด้วยแผนภาพ/trace
- สอบกลางภาค: สอบออนไลน์จำกัดเวลา เน้น “อธิบายและ trace”
- สอบปลายภาค: สอบในห้องสอบ เน้น “อธิบายและ trace” ไม่เน้นจำสูตรอย่างเดียว รวมทุกหัวข้อโจทย์ระยะคน
- ไม่มีการบ้านเขียนโปรแกรม (No programming assignments) เป็นข้อบังคับของรายวิชา
การประเมินผลจะยึด **pseudocode + trace + แผนภาพ + การอธิบายเหตุผล + Big-O** เป็นหลัก

5 การประเมินผล (Assessment)

5.1 สัดส่วนคะแนน (ภาพรวมการตัดเกรด)

องค์ประกอบ	%	คำอธิบายสั้น ๆ
พฤติกรรมเข้าเรียน/ส่งงาน	10	เข้าเรียน ตรงเวลา ส่งงานย่อย/กิจกรรมในคาบ
การบ้าน	10	ใบงาน/การบ้าน เน้น trace + อธิบาย + Big-O
สอบย่อย	15	2 ครั้ง สอบ takehome 1 สัปดาห์
งานกลุ่ม	15	วิเคราะห์โจทย์ + เลือก DS/algorithm + สื่อสารเป็นระบบ
สอบกลางภาค	20	สอบออนไลน์จำกัดเวลา
สอบปลายภาค	30	ครอบคลุมทั้งรายวิชา เน้นการประยุกต์และการอธิบาย

หมายเหตุ: สอบย่อยคะแนนเต็มจริงครั้งละ 10 คะแนนรวม 20 คะแนน cap ที่ 15 คะแนน

ในการสอบปลายภาคและกลางภาคจะให้คะแนนเกินไว้อย่างละ 5 คะแนนและตัดเหลือเท่าที่เก็บสำหรับคนที่ได้เกิน

5.2 เกณฑ์ตัดเกรด (Grading Scale)

เกรด	ช่วงคะแนนรวม
A	80--100
B+, B, C+, C, D+, D	อิงกลุ่มตามเส้นโค้งปกติ
F	ต่ำกว่า 50

6 การสอบออนไลน์ หรือสอบ takehome แบบ Open book (ถ้ามีการประกาศใช้)

6.1 รูปแบบที่แนะนำ

- **Open book:** เปิดเน็ต/เอกสารได้ (แต่ต้องอธิบายได้ด้วยตนเอง)
- คำตอบหลักเน้น แผนภาพ + trace + อธิบายเหตุผล (มากกว่าการท่องจำ)

6.2 มาตรฐานการส่งไฟล์ (Submission Standard)

- รวมเป็น PDF ไฟล์เดียว และตั้งชื่อไฟล์ตามรูปแบบที่ประกาศ
- เขียน ชื่อ--รหัส--เลขหน้า ในทุกหน้า และระบุเลขข้อให้ชัด
- รูป/สแกนต้อง อ่านได้ชัด นักศึกษารับผิดชอบความครบถ้วนด้วยตนเอง

7 นโยบายรายวิชา (Course Policies)

7.1 การเข้าเรียนและการส่งงาน

- รายวิชานี้พึ่งพากิจกรรมในคาบและการฝึก trace จึงแนะนำให้เข้าเรียนสม่ำเสมอ
- การบ้าน/ใบงานควรส่งตามกำหนดเพื่อให้ได้รับ feedback ทันรอบการเรียนรู้

7.2 การส่งงานล่าช้า (Late Policy)

- นักศึกษามีโควต้าส่งงานช้าได้คนละ 4 วัน ตลอดภาค โดยไม่ต้องแจ้งเหตุผล (นับเป็นรายวัน ปิดขึ้นเสมอ)
- หลังหมดโควต้า งานที่ส่งช้าจะไม่ได้รับคะแนน เว้นแต่มีเหตุสุดวิสัยและแจ้งผู้สอนพร้อมหลักฐานโดยเร็ว

7.3 การขอตรวจทานคะแนน (Regrade Request)

หากต้องการขอตรวจทานคะแนน ให้ส่งคำร้องภายใน 7 วัน หลังประกาศคะแนนชิ้นงานนั้น โดยอธิบาย ข้อที่ต้องการทบทวน + เหตุผลเชิงเกณฑ์ให้คะแนน อย่างชัดเจน

8 นโยบายความซื่อสัตย์ทางวิชาการและการใช้ AI

รายวิชานี้ให้ความสำคัญกับ ความเข้าใจและความรับผิดชอบต่อคำตอบของตนเอง

- อนุญาตให้ใช้เครื่องมือช่วย (รวมถึง AI) เพื่อ ช่วยอธิบาย/ช่วยตรวจสอบความเข้าใจ ได้
- แต่คำตอบที่ส่งต้องเป็น ส่วนของผู้เรียน และผู้เรียนต้องสามารถ อธิบายขั้นตอน/trace/เหตุผล ได้จริง
- หากคัดลอกคำตอบโดยไม่อ้างอิง หรือทุจริต จะดำเนินการตามระเบียบของคณะ/มหาวิทยาลัย

9 แผนรายสัปดาห์ (16 ครั้งเรียน)

9.1 ภาพรวมหัวข้อและทักษะที่ฝึก

สัปดาห์	หัวข้อหลัก	ทักษะที่ฝึก
1	Intro + Memory + ADT	เหตุผล ที่ ต้อง มี DS, ภาพ memory, แยก ADT vs implementation
2	Pseudocode + Big-O	อ่าน/เขียน pseudocode, best/worst case, $O(\cdot)$
3	Arrays + Binary Search	ภาพ array ใน memory, trace binary search
สอบย่อย 1 (week 1 - 2)		
4	Dynamic Array + Singly Linked List	เปรียบเทียบ operation, trace insert/delete
5	Doubly/Circular Linked List	pointer update checklist, กรณีขอบ
6	Stack + Queue	LIFO/FIFO, simulate ด้วยภาพ, expression evaluation (แนวคิด)
7	Recursion	base/recursive case, call stack, divide-and-conquer (intro)
Week 8 สอบกลางภาค (week 3 - 6 including week 1-2, 7 for bonus)		
9	Trees + Traversal	root/leaf/height, traversal แบบ BFS/DFS
10	BST	insert/search/delete (3 cases), trace อย่างเป็นขั้น
11	Tries	prefix matching, startsWith, trace search
12	Priority Queue + Heap	sift up/down, heap sort (trace)
13	Hash Table	hash function, collision (chaining/open addressing)
สอบย่อย 3 (week 9 - 12)		
14	Graphs	representation, BFS/DFS, shortest path/ MST (แนวคิด+ตัวอย่างเล็ก)
15	Searching & Sorting	เปรียบเทียบ algorithms, Big-O, trade-off
16	Review + Final Prep	mixed problems: เลือก DS/algorithm + trace + อธิบายเหตุผล
สอบปลายภาค (แบบ comprehension รวมทุกสัปดาห์)		

หมายเหตุ: กำหนดการสอบปลายภาคจัดตามปฏิทินมหาวิทยาลัย และรายละเอียดจะแจ้งผ่าน Teams

10 เอกสารและทรัพยากรประกอบ

10.1 เอกสารหลัก/เอกสารประกอบ (อาจปรับตามประกาศ)

- เอกสารประกอบการสอนและใบงานของรายวิชา (อัปโหลดรายสัปดาห์ใน Teams)
- เอกสารอ้างอิงเพิ่มเติม (ไม่บังคับ):
 - Data Structures the Fun Way* (Jeremy Kubica)
 - Data Structures and Algorithms in Python* (Goodrich et al.)

10.2 สิ่งที่ต้องเตรียม (Recommended)

- Mindset แบบ “ทดและวางแผนก่อนลงมือเขียน code เสมอ ดีกว่าเขียนไปเรื่อย ๆ จนแก้ยาก”

11 การสื่อสารและการขอความช่วยเหลือ

11.1 ช่องทางติดต่อ

- ประกาศรายวิชา/ไฟล์เรียน/การส่งงาน: Microsoft Teams (รวมถึงแชทส่วนตัว)
- อีเมลผู้สอน: phaphontee@sau.ac.th

หมายเหตุ: Syllabus อาจปรับเล็กน้อยตามจังหวะห้องเรียน/วันหยุด โดยจะแจ้งล่วงหน้าผ่านช่องทางประกาศรายวิชา